
IES RAFAEL ALBERTI

Ciclo Formativo de Grado Superior — Informática

Análisis y Predicción de Lanzamientos Espaciales con Big Data y BI

Trabajo de Fin de Ciclo

Alumno: Juan Manuel Vega Carrillo
Correo: juanmavegacarrillo@gmail.com
Centro: IES Rafael Alberti
Curso: 2025 / 2026
Fecha: May de 2026

Resumen

Plataforma integral de análisis de datos para lanzamientos espaciales históricos. Integra tres APIs públicas (Launch Library 2, SpaceX API y Open-Meteo) mediante un pipeline Docker automatizado: la ingesta en Python extrae y persiste los datos en formato JSONL; Apache Spark 3.5 los transforma en capas Silver y Gold (arquitectura *medallion*); y un dashboard Streamlit con ocho paneles interactivos permite explorarlos con mapas 3D, animaciones y un simulador de probabilidad de éxito. Todo el sistema arranca con `docker compose up`.

Palabras clave: Big Data, Apache Spark, Streamlit, Docker, Arquitectura Medallion, Launch Library 2, SpaceX API, Parquet, Business Intelligence.

Índice

1. Introducción	2
2. Arquitectura del Sistema	2
2.1. Flujo de datos con Docker Compose	3
2.2. Arquitectura Medallion	3
3. Implementación	3
3.1. Capa de Ingesta (Python 3.10)	3
3.2. Capa de Procesamiento (Apache Spark 3.5.1)	4
3.3. Capa de Visualización (Streamlit + Plotly)	4
4. Dashboard Interactivo	4
5. Resultados	4
5.1. Stack tecnológico	5
6. Conclusiones	6
Referencias	7

1. Introducción

Desde el lanzamiento del *Sputnik* en 1957 hasta los cohetes reutilizables de SpaceX en la actualidad, la humanidad ha acumulado décadas de datos sobre misiones espaciales. Sin embargo, esta información se encuentra dispersa en múltiples fuentes y formatos, dificultando su análisis integrado.

El objetivo de este proyecto es construir una plataforma Big Data *end-to-end* que integre, procese y visualice esos datos, permitiendo responder preguntas como: ¿qué agencia tiene más lanzamientos? ¿cómo afecta el viento al éxito de una misión? ¿qué cohete ofrece mayor fiabilidad?

Objetivos principales

- Integrar **3 APIs públicas** de datos espaciales en un único pipeline de datos.
- Implementar la **arquitectura medallion** (Raw → Silver → Gold) con Apache Spark.
- Construir un **dashboard interactivo** con 8 paneles temáticos: histórico, agencias, cohetes, clima, mapa 3D, galería y simulador.
- Contenedorizar todo el sistema con **Docker Compose** para despliegue con un único comando.

2. Arquitectura del Sistema

El sistema se estructura en tres capas diferenciadas, implementadas como servicios Docker que se encadenan automáticamente:

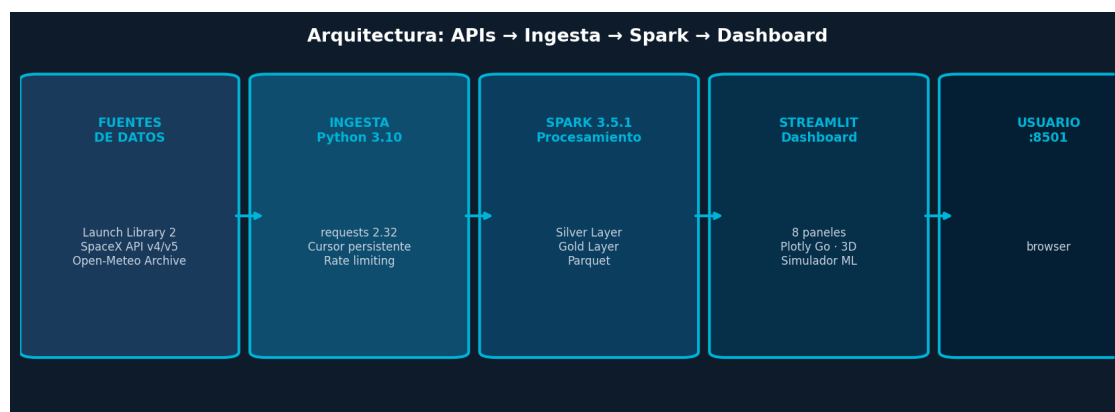


Figura 1: Arquitectura general: APIs → Ingesta → Spark → Dashboard

2.1. Flujo de datos con Docker Compose

Paso	Servicio	Imagen	Acción
1	ingestion	python:3.10-slim	Extrae APIs y guarda JSONL en data/raw/
2	processing	apache/spark:3.5.1	Transforma JSONL a Parquet Silver/Gold
3	dashboard	python:3.11-slim	Arranca Streamlit en localhost:8501

Los pasos 2 y 3 usan la condición `service_completed_successfully`, de forma que cada servicio espera a que el anterior haya terminado con éxito antes de arrancar.

2.2. Arquitectura Medallion

Capa	Formato	Contenido
RAW	JSONL versionado	launch_library_launches.jsonl, spacex_rockets.json, open_meteo_samples.jsonl, manifest.json
SILVER	Parquet particionado	launches/, weather/, spacex_rockets/, images/
GOLD	Parquet agregado	company_year_metrics (KPIs por agencia/año), launch_features (fea- ture store ML)

3. Implementación

3.1. Capa de Ingesta (Python 3.10)

El módulo `ingestion/src/main.py` se encarga de la extracción desde las tres fuentes:

Fuentes de datos integradas

- **Launch Library 2** (11.thespacedevs.com/2.2.0/launch/): lanzamientos históricos con fecha, estado, agencia, cohete y sitio. Usa *cursor persistente* para ingesta incremental y manejo de rate limit (429) con activación de modo sintético.
- **SpaceX API v4/v5** (api.spacexdata.com): cohetes, cargas útiles, núcleos reutilizables e imágenes Flickr. Requiere POST con objeto de consulta en v5.
- **Open-Meteo Archive** (archive-api.open-meteo.com): temperatura media y viento máximo por coordenadas/fecha. Throttling de 0.5s entre peticiones.

La estrategia de reintentos sigue backoff exponencial ($2s \rightarrow 4s \rightarrow 8s$). Cada ejecución genera una carpeta `data/raw/YYYYMMDD_HHMMSS/` con todos los ficheros y un `manifest.json` con metadatos de la extracción.

3.2. Capa de Procesamiento (Apache Spark 3.5.1)

El job `processing/src/silver_gold.py` lee el directorio `raw` más reciente y aplica transformaciones en dos fases:

1. **Silver Layer:** normalización de tipos (`cast`, timestamps, flags binarios), particionado por año.
2. **Gold Layer:** agregaciones SQL (`GROUP BY provider_name, launch_year`) para KPIs y JOIN de lanzamientos con clima e imágenes para el feature store de ML.

Spark corre en modo `local[2]` dentro del contenedor, por lo que no requiere un clúster externo.

3.3. Capa de Visualización (Streamlit + Plotly)

El dashboard está implementado en `dashboard/app.py` (1 783 líneas). Usa `@st.cache_data` para cachear las lecturas Parquet y CSS inyectado para el tema espacial oscuro con 200 estrellas SVG animadas.

Panel	Nombre	Contenido
1	Resumen Global	7 KPIs: total, tasa éxito, agencias, cohetes, años
2	Histórico	Heatmap mes × año + tendencia anual
3	Proveedores	Ranking + carrera animada de barras por año
4	Cohetes	Métricas SpaceX, comparativa éxito/fallo
5	Clima	Dispersión temperatura y viento vs. resultado
6	Mapa Global	Globo 3D ortográfico (Scattergeo Plotly)
7	Galería	Grid de imágenes con filtros y paginación
8	Simulador	$P(\text{éxito}) = \text{base_rate} \times \text{temp_factor} \times \text{wind_factor}$

4. Dashboard Interactivo

Acceso al dashboard

Con el sistema en marcha (`docker compose up`), el dashboard está disponible en <http://localhost:8501>. La primera vez tarda ~10 min (ingesta de APIs incluida); las siguientes arranca en segundos.

5. Resultados

El pipeline ha procesado **1,000 registros** de lanzamientos (1957–2023), incluyendo 10 agencias espaciales. La tasa global de éxito es del **85.1 %**.

Las tres agencias con mayor actividad son **SpaceX** (500 lanzamientos), **United States Air Force** (268) y **Soviet Space Program** (129).

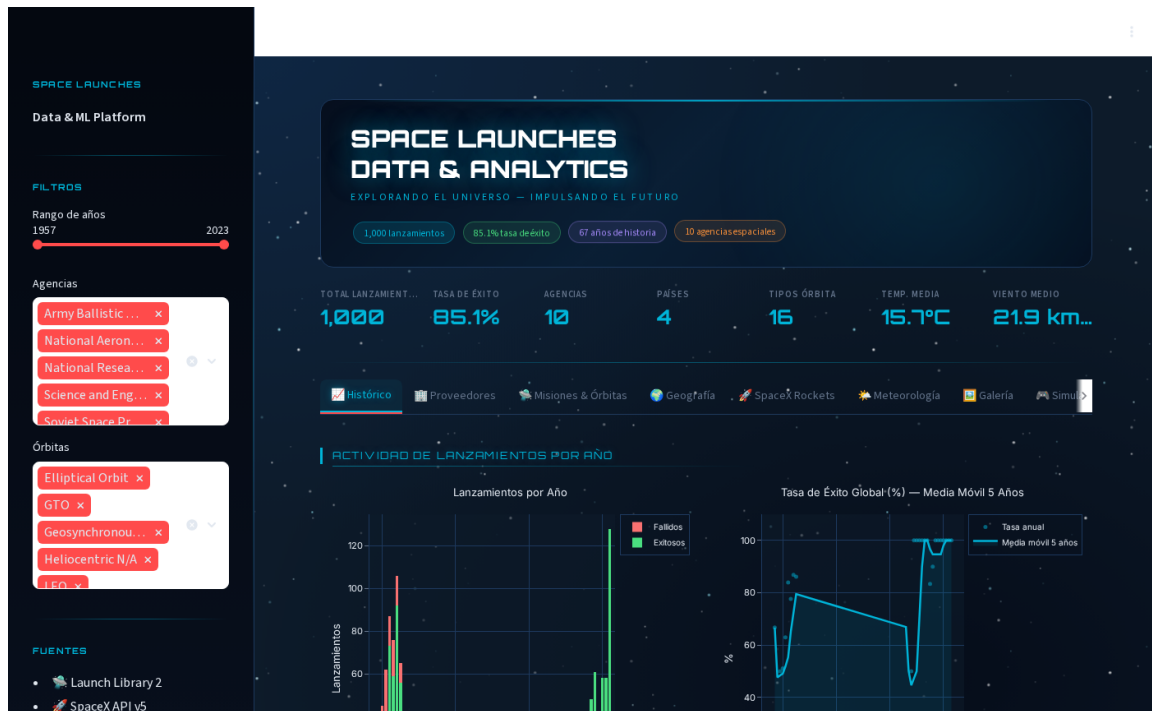
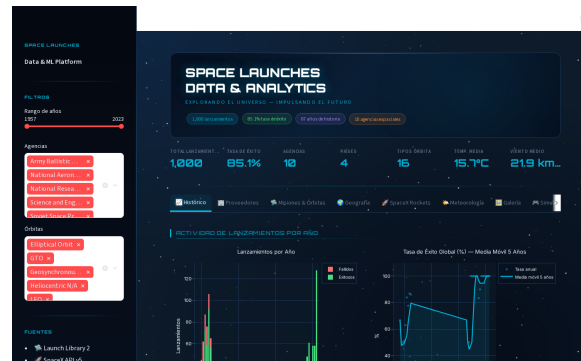


Figura 2: Vista general del dashboard con tema espacial oscuro

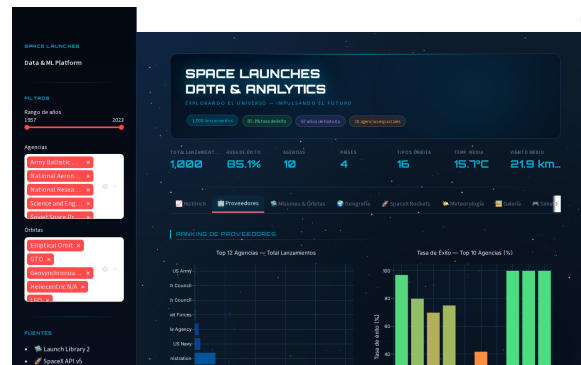


(a) Panel 2: Histórico — Heatmap estacional

5.1. Stack tecnológico

Capa	Tecnología	Versión
Ingesta	Python + requests	3.10 / 2.32
Procesamiento	Apache Spark + PySpark	3.5.1
Almacenamiento	Parquet (Silver/Gold)	—
Visualización	Streamlit + Plotly Go	1.57 / 6.7
Infraestructura	Docker Compose	28.5 / v2.40
Base de datos	PostgreSQL (opcional)	16-alpine

6. Conclusiones



(a) Panel 3: Proveedores — Carrera animada

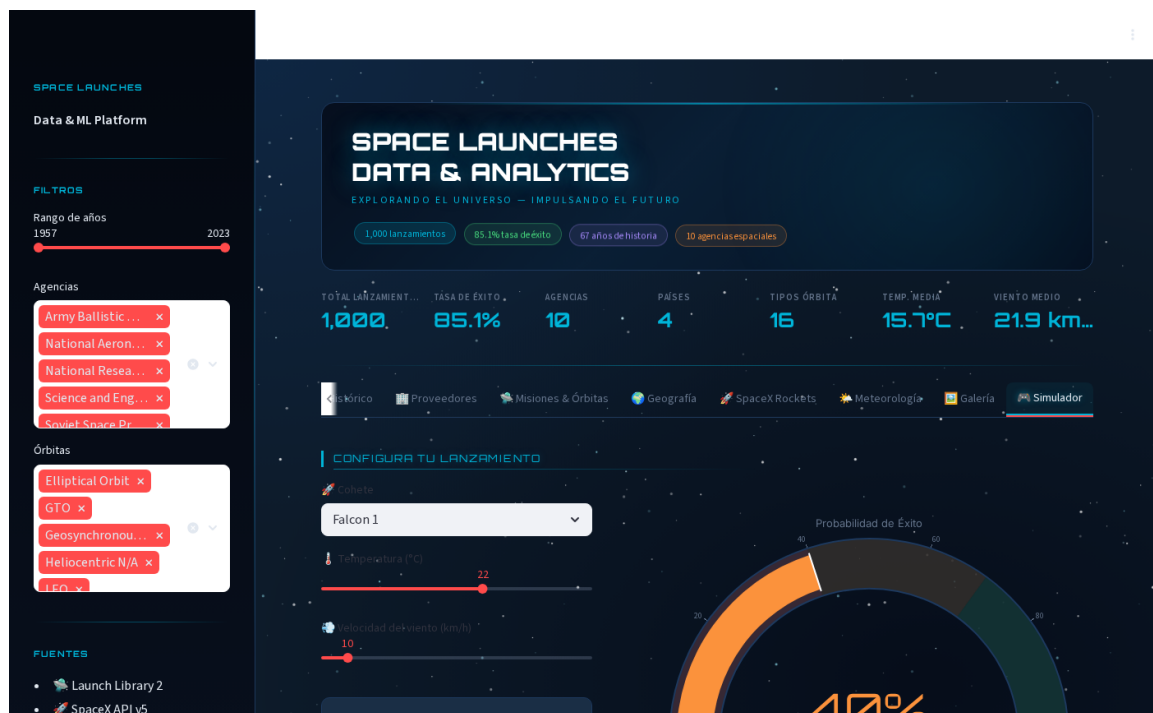


Figura 5: Panel 8: Simulador de probabilidad de éxito

Logros del proyecto

- **Arquitectura Medallion implementada** con Apache Spark: pipeline Raw → Silver → Gold reproducible y trazable.
- **Despliegue con un comando** (docker compose up): desde cero hasta el dashboard funcional en ~10 minutos.
- **Dashboard con 8 paneles** incluyendo globo 3D, carrera animada de barras, galería de imágenes y simulador estadístico.
- **1,000 registros** procesados de 10 agencias espaciales (1957–2023), tasa de éxito del 85.1 %.



Figura 6: Métricas principales del conjunto de datos

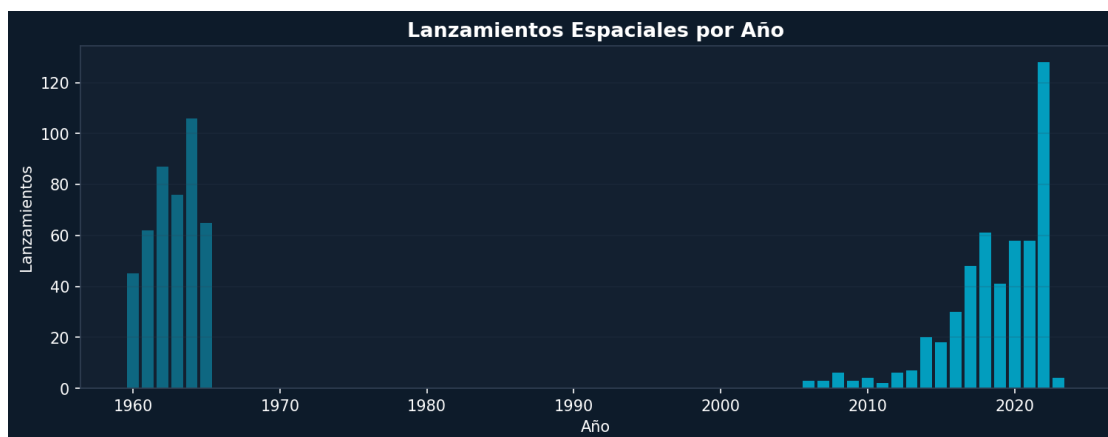


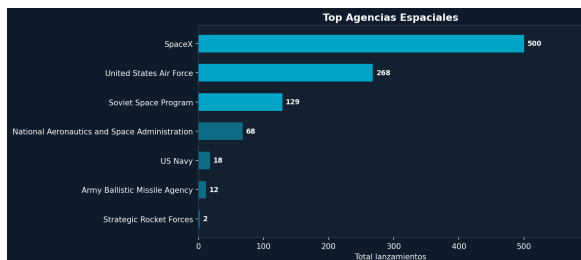
Figura 7: Distribución de lanzamientos por año (1960–2023)

Líneas de trabajo futuro

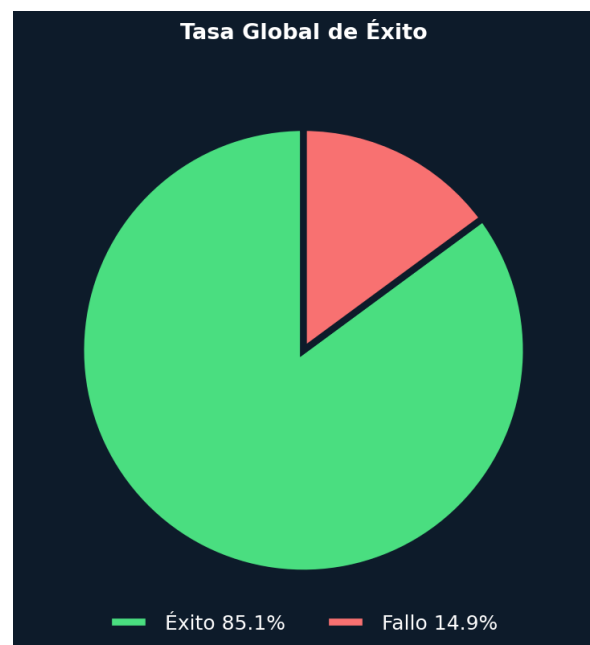
- Modelo ML real (XGBoost/RandomForest) para el simulador de éxito.
- Ingesta en tiempo real con Spark Structured Streaming.
- Ampliar al histórico completo de Launch Library (>5 000 lanzamientos).
- Integrar alertas automáticas para misiones próximas.

Referencias

1. The Space Devs. *Launch Library 2 API*. <https://thespacedevs.com/llapi>
2. r-spacex. *SpaceX-API v4/v5*. <https://github.com/r-spacex/SpaceX-API>
3. Open-Meteo. *Historical Weather API*. <https://open-meteo.com>
4. Apache Software Foundation. *Apache Spark 3.5 Docs*. <https://spark.apache.org/docs/3.5.1/>
5. Streamlit Inc. *Streamlit Documentation*. <https://docs.streamlit.io>
6. Docker Inc. *Compose Specification*. <https://docs.docker.com/compose/compose-file/>
7. Databricks. *The Medallion Architecture*. <https://www.databricks.com/glossary/medallion-architecture>



(a) Top agencias espaciales por lanzamientos



(b) Tasa global de éxito